

TYPESCRIPT

MEDIUM

Generic TTL Cache

Your team's backend makes expensive calls to a pricing service. You need a small in-memory cache that can store values of ANY type (strings, numbers, objects) keyed by string, where each entry automatically expires after a given time-to-live (TTL) in milliseconds.

```
interface CacheEntry<T> {
  value: T;
  expiresAt: number;
}

class TypedCache<T> {
  private store = new Map<string, CacheEntry<T>>();

  set(key: string, value: T, ttlMs: number): void {
    // TODO: implement
  }

  get(key: string): T | undefined {
    // TODO: implement
  }
}
```

Your task:

1. Implement `set` so it stores the value with an expiry timestamp computed from `ttlMs`.
2. Implement `get` so it returns `undefined` if the key is missing OR if the entry has expired (and delete expired entries when found).
3. Add a `has(key: string): boolean` method that reuses `get` to check freshness without duplicating expiry logic.

MEDIUM

API Response Renderer

A frontend component needs to render three possible states of a data fetch: still loading, successfully loaded with data, or failed with an error message. You want the type system to guarantee every state is handled before the code compiles.

```
type ApiResponse<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

function renderResponse<T>(response: ApiResponse<T>, formatData: (data: T) => string): string {
  // TODO: implement using a switch on response.status
}
```

Your task:

1. Implement `renderResponse` using a `switch` on `response.status` that returns an appropriate string for each state.
2. For the `success` case, call `formatData` on `response.data`.
3. Add a `default` branch that assigns the remaining value to a variable typed `never`, so that if a new state is ever added the code fails to compile.

TYPESCRIPT
MEDIUM

Partial Profile Update

Your app's 'edit profile' form lets a user change their name, email, age, and bio, but never their `id`. You want a single update function whose type signature makes it impossible to accidentally overwrite the `id` field, while allowing any subset of the other fields to be updated.

```
interface UserProfile {
  id: string;
  name: string;
  email: string;
  age: number;
  bio: string;
}

function updateProfile(profile: UserProfile, updates: /* TODO: type */): UserProfile {
  // TODO: implement
}
```

Your task:

1. Write the type for `updates` using built-in utility types so it only allows an optional subset of `name`, `email`, `age`, and `bio` (never `id`).
2. Implement `updateProfile` to merge `updates` into `profile` and return the merged result.
3. Ensure the return type is still a full `UserProfile` (no fields become optional in the output).

MEDIUM

Retry With Backoff

Network calls to a flaky downstream service occasionally fail. You need a generic retry helper that re-runs an async operation up to N times with a delay between attempts, and preserves the original operation's return type end-to-end.

```
async function retry<T>(  
  fn: () => Promise<T>,  
  attempts: number,  
  delayMs: number  
) : Promise<T> {  
  // TODO: implement  
}
```

Your task:

1. Implement `retry` so it calls `fn` up to `attempts` times, returning the first successful result.
2. Wait `delayMs` milliseconds between failed attempts (but not after the final attempt).
3. If every attempt fails, re-throw the last error captured.

MEDIUM

Payment Method Narrowing

A checkout page accepts three payment method shapes, each with different fields. You need to safely narrow the union to display a masked summary string without ever accessing a field that doesn't exist on the current variant.

```
type PaymentMethod =
  | { type: 'credit_card'; cardNumber: string; cvv: string }
  | { type: 'paypal'; email: string }
  | { type: 'bank_transfer'; accountNumber: string; routingNumber: string };

function maskPaymentInfo(method: PaymentMethod): string {
  // TODO: implement, showing only the last 4 digits for card/account numbers
}
```

Your task:

1. Write a user-defined type guard `isCreditCard(method: PaymentMethod)` using an `is` predicate.
2. Use the guard (or discriminant narrowing) inside `maskPaymentInfo` to branch on all three variants.
3. For `credit_card`, return a string showing only the last 4 digits of `cardNumber`; similarly mask `accountNumber` for `bank_transfer`.

MEDIUM

Fluent Query Builder

You're building a tiny internal query-builder library so engineers can write `new QueryBuilder('users').where("age > 18").limit(10).build()` instead of hand-writing SQL strings. Method chaining must keep working through subclasses without hardcoding the return type.

```
interface QueryState {
    table: string;
    filters: string[];
    limitValue?: number;
}

class QueryBuilder {
    private state: QueryState;

    constructor(table: string) {
        this.state = { table, filters: [] };
    }

    where(condition: string): this {
        // TODO: implement
    }

    limit(n: number): this {
        // TODO: implement
    }

    build(): string {
        // TODO: implement
    }
}
```

Your task:

1. Implement ``where`` to push a condition and return ``this`` for chaining.
2. Implement ``limit`` to set the limit value and return ``this``.
3. Implement ``build`` to assemble a SQL-like string: ``SELECT * FROM <table>`` plus an optional ``WHERE ... AND ...`` clause and an optional ``LIMIT <n>`` clause.

MEDIUM

Strongly Typed Event Emitter

Your app currently uses Node's untyped `EventEmitter`, and it keeps causing bugs where a listener expects the wrong payload shape. You want an emitter where the event name determines the payload type at compile time, for a fixed set of app events (`login`, `logout`, `error`).

```
interface EventMap {
    login: { userId: string };
    logout: { userId: string };
    error: { message: string; code: number };
}

class TypedEmitter<T extends object> {
    private listeners: { [K in keyof T]?: Array<(payload: T[K]) => void> } = {};

    on<K extends keyof T>(event: K, handler: (payload: T[K]) => void): void {
        // TODO: implement
    }

    emit<K extends keyof T>(event: K, payload: T[K]): void {
        // TODO: implement
    }
}
```

Your task:

1. Implement `on` to register a handler for the given event, lazily initializing the listener array.
2. Implement `emit` to call all registered handlers for that event with the payload.
3. Verify (by writing an example usage) that calling `emitter.on('login', handler)` types `handler`'s parameter as `{ userId: string }` automatically.

MEDIUM

Generic Stack

You're implementing an 'undo' feature for a text editor. It needs a simple, type-safe stack of editor snapshots (or any other type callers choose), supporting push/pop/peek and reporting whether it's empty.

```
class Stack<T> {
    private items: T[] = [];

    push(item: T): void {
        // TODO: implement
    }

    pop(): T | undefined {
        // TODO: implement
    }

    peek(): T | undefined {
        // TODO: implement
    }
}
```

Your task:

1. Implement `push`, `pop`, and `peek`.
2. Add a read-only `size` accessor (getter) returning the current number of items.
3. Add an `isEmpty()`: boolean method.

TYPESCRIPT
MEDIUM

Typed Config Parser

Your app reads configuration values from environment variables, which always arrive as raw strings. You want a single `parseConfigValue` function whose return type automatically matches the requested target type (`number`, `boolean`, or `string`), so callers don't need manual casts.

```
// TODO: add overload signatures so the return type depends on `type`
function parseConfigValue(value: string, type: 'number' | 'boolean' | 'string'): number | boolean | string {
  // TODO: implement
}
```

Your task:

1. Add three overload signatures: one each for `type: 'number'` (returns `number`), `type: 'boolean'` (returns `boolean`), and `type: 'string'` (returns `string`).
2. Implement the single implementation signature that satisfies all three overloads.
3. Ensure `parseConfigValue('true', 'boolean')` is inferred as `boolean` at the call site (not `number | boolean | string`).

MEDIUM

Deep Readonly Settings

Application settings are loaded once at startup and must never be mutated afterward, including nested objects like `notifications`. TypeScript's built-in `readonly<T>` only protects the top level. You need a `DeepReadonly<T>` mapped type that recursively locks nested objects too.

```
interface AppSettings {
  theme: string;
  notifications: {
    email: boolean;
    sms: boolean;
  };
}

type DeepReadOnly<T> = /* TODO: define recursively using a mapped type */;

function freezeSettings(settings: AppSettings): DeepReadOnly<AppSettings> {
  // TODO: implement
}
```

Your task:

1. Define ``DeepReadOnly<T>`` as a mapped type that makes every property ``readonly``, recursing into properties that are themselves objects.
2. Implement ``freezeSettings`` to return the settings typed as ``DeepReadOnly<AppSettings>``.
3. Write a short comment or example showing that ``result.notifications.email = false`` would now be a compile error.

MEDIUM

Async Return Type Extractor

You have many `async` service functions across the codebase and want a reusable utility type that extracts the resolved data type from any of them, similar to how TypeScript's built-in `ReturnType` works but unwrapping the `Promise`.

```
type AsyncReturnType<T> extends (...args: any[]) => Promise<any> = /* TODO: conditional type using infer */;

async function fetchUser(id: string): Promise<{ id: string; name: string }> {
  return { id, name: 'Alice' };
}

type User = /* TODO: use AsyncReturnType here */;
```

Your task:

1. Define ``AsyncReturnType<T>`` using a conditional type with ``infer`` to extract the resolved value type from a function returning a ``Promise``.
2. Use it to derive ``User`` from ``typeof fetchUser`` without manually writing out the ``{ id, name }`` shape.
3. Write a function ``printUser(user: User): void`` that uses the derived type to prove it resolved correctly.

TYPSCRIPT

MEDIUM

Order Status Transitions

An e-commerce order moves through a strict lifecycle: pending, shipped, delivered, or cancelled — and each state carries different extra data (e.g. `shipped` has a tracking number). You need a `nextState` function that advances an order to its next logical state, fully typed so invalid states are impossible to construct.

```
type OrderState =
  | { status: 'pending' }
  | { status: 'shipped'; trackingNumber: string }
  | { status: 'delivered'; deliveredAt: Date }
  | { status: 'cancelled'; reason: string };

function nextState(state: OrderState): OrderState {
  // TODO: implement: pending -> shipped -> delivered; delivered/cancelled stay as-is
}
```

Your task:

1. Implement `nextState` using a `switch` on `state.status`.
2. `pending` should transition to `shipped` (invent a tracking number), `shipped` should transition to `delivered` (using the current time).
3. `delivered` and `cancelled` are terminal — return the state unchanged.

MEDIUM

Sort Items By Key

A product listing page needs to sort arrays of objects (products, users, orders — any shape) by a given property name chosen at the call site, e.g. `sortByKey(products, 'price')`. The property name must be validated against the object's actual keys at compile time.

```
function sortByKey<T, K extends keyof T>(items: T[], key: K): T[] {
  // TODO: implement, supporting both number and string field comparisons
}
```

Your task:

1. Constrain the generic so `key` can only be a valid property name of `T` (using `keyof`).
2. Implement the sort without mutating the original array.
3. Handle both numeric fields (numeric comparison) and string fields (locale-aware string comparison) correctly.

TYPSCRIPT

MEDIUM

HTTP Status Lookup Table

You're building a small helper that maps HTTP status codes to human-readable messages for logging. The table should be type-safe (keys are numbers, values are strings) and the lookup function must gracefully handle unknown codes.

```
type HttpStatusMessages = /* TODO: type this as a lookup from number to string */;

const statusMessages: HttpStatusMessages = {
  200: 'OK',
  404: 'Not Found',
  500: 'Internal Server Error',
};

function getStatusMessage(messages: HttpStatusMessages, code: number): string {
  // TODO: implement, returning 'Unknown Status' for missing codes
}
```

Your task:

1. Define `HttpStatusMessages` using `Record<number, string>` (or an equivalent index signature).
2. Implement `getStatusMessage` to look up the code and fall back to `Unknown Status` when not present.
3. Use the nullish coalescing operator (`??`) rather than a manual `if` check for the fallback.

TYPESCRIPT

MEDIUM

Paginated Data Generator

An admin dashboard needs to stream through every record of a paginated API (which returns a page of items plus a `nextCursor`) without loading everything into memory at once. You want an async generator that callers can iterate with `for await...of`.

```
interface Page<T> {
  items: T[];
  nextCursor: string | null;
}

async function* paginate<T>({
  fetchPage: (cursor: string | null) => Promise<Page<T>>
}): AsyncGenerator<T, void, unknown> {
  // TODO: implement, fetching pages until nextCursor is null
}
```

Your task:

1. Implement `paginate` to repeatedly call `fetchPage` starting with a `null` cursor, `yield`-ing each item from each page.
2. Continue fetching subsequent pages using the returned `nextCursor` until it is `null`.
3. Ensure the function's declared type correctly reflects that it yields individual items of type `T`, not whole pages.

TYPESCRIPT
MEDIUM

Typed Counter Reducer

You're wiring up Redux-style state management for a counter widget that supports incrementing, decrementing by a variable amount, and resetting. The reducer must be fully typed so dispatching an invalid action shape is a compile error.

```
interface CounterState {
  count: number;
}

type CounterAction =
  | { type: 'increment'; by: number }
  | { type: 'decrement'; by: number }
  | { type: 'reset' };

function counterReducer(state: CounterState, action: CounterAction): CounterState {
  // TODO: implement
}
```

Your task:

1. Implement `counterReducer` using a `switch` over `action.type`.
2. `increment`/`decrement` should adjust `count` by `action.by`; `reset` should set `count` back to 0.
3. Return a new state object each time rather than mutating the input `state`.

MEDIUM

Service Registry Singleton

Your app has several cross-cutting services (logger, analytics client, feature-flag client) that should be instantiated once and looked up by name anywhere in the codebase, with type-safe registration and resolution.

```
class ServiceRegistry {
    private static instance: ServiceRegistry;
    private services = new Map<string, unknown>();

    private constructor() {}

    static getInstance(): ServiceRegistry {
        // TODO: implement singleton access
    }

    register<T>(key: string, service: T): void {
        // TODO: implement
    }

    resolve<T>(key: string): T {
        // TODO: implement, throwing if not found
    }
}
```

Your task:

1. Implement `getInstance`` so the class is a true singleton (only one instance ever created).
2. Implement `register`` to store a service of any type under a string key.
3. Implement `resolve<T>`` to retrieve and cast the stored service back to `T``, throwing a descriptive error if the key is missing.

TYPESCRIPT

MEDIUM

Safe JSON Product Parser

Your service receives product data as raw JSON strings from an untrusted external partner feed. Before using the data, you must validate its shape at runtime — `JSON.parse` alone only gives you `any`, which is unsafe to trust blindly.

```
interface Product {
  id: string;
  name: string;
  price: number;
}

function isProduct(value: unknown): value is Product {
  // TODO: implement runtime shape validation
}

function parseProduct(json: string): Product | null {
  // TODO: implement using JSON.parse + isProduct, returning null on any failure
}
```

Your task:

1. Implement `isProduct` as a type guard that checks, at runtime, that `value` has `id` (string), `name` (string), and `price` (number).
2. Implement `parseProduct` to safely `JSON.parse` the input, catching parse errors and returning `null` on failure.
3. Use `isProduct` to validate the parsed result before returning it, returning `null` if validation fails.

TYPSCRIPT

MEDIUM

Nested Config Resolver

An app's configuration object is loaded from a partially-filled JSON file, so every nested section is optional. You need helper functions that read specific settings and fall back to sensible defaults when any part of the path is missing.

```
interface AppConfig {
  api?: {
    baseUrl?: string;
    timeout?: number;
  };
  featureFlags?: {
    darkMode?: boolean;
  };
}

function getBaseUrl(config: AppConfig): string {
  // TODO: implement, default 'https://api.default.com'
}

function getTimeout(config: AppConfig): number {
  // TODO: implement, default 5000
}
```

Your task:

1. Implement `getBaseUrl` and `getTimeout` using optional chaining (`?.`) to safely traverse the nested optional structure.
2. Use nullish coalescing (`??`) to supply the defaults, only falling back on `null`/`undefined` (not on falsy-but-valid values like `0`).
3. Add a third function `isDarkModeEnabled(config: AppConfig): boolean` defaulting to `false`.

MEDIUM

Tree Search Utility

A file-explorer UI represents folders and files as a nested tree. You need a generic, reusable function that searches any such tree (regardless of what data it holds) for the first node matching a predicate, e.g. finding a file by name.

```
interface TreeNode<T> {
    value: T;
    children: TreeNode<T>[];
}

function findInTree<T>(node: TreeNode<T>, predicate: (value: T) => boolean): TreeNode<T> | null {
    // TODO: implement depth-first search
}
```

Your task:

1. Implement `findInTree` as a depth-first search: check the current node first, then recurse into children in order.
2. Return the first matching `TreeNode<T>` found, or `null` if nothing matches anywhere in the tree.
3. Ensure the function type-checks generically for any `T` (strings, numbers, custom objects) without using `any`.

MEDIUM

Typed API Route Builder

Your frontend calls a REST API with routes shaped like ``v1/users`` or ``v2/orders``. You want the compiler to reject any route string that doesn't match this exact versioned-resource pattern, catching typos like ``v1/usres`` before runtime.

```
type ApiVersion = 'v1' | 'v2';
type Resource = 'users' | 'orders' | 'products';

type ApiRoute = /* TODO: template literal type combining ApiVersion and Resource */;

function buildRoute(version: ApiVersion, resource: Resource): ApiRoute {
  // TODO: implement
}
```

Your task:

1. Define ``ApiRoute`` as a template literal type of the form ``/${ApiVersion}/${Resource}``.
2. Implement ``buildRoute`` to construct and return a valid ``ApiRoute`` from its parts.
3. Write a ``callApi(route: ApiRoute): string`` function and show that passing a malformed literal string would be rejected (as a comment).

MEDIUM

Typed Pipe Utility

You keep writing chains like ``exclaim(toUpperCase(shout(text)))`` for text transformations and want a ``pipe`` helper that composes two functions left-to-right, with the compiler correctly inferring the intermediate and final types.

```
function pipe<A, B, C>(fn1: (a: A) => B, fn2: (b: B) => C): (a: A) => C {
  // TODO: implement
}
```

Your task:

1. Implement ``pipe`` to return a new function that applies ``fn1`` then ``fn2`` in sequence.
2. Demonstrate usage by composing a ``toUpperCase: (s: string) => string`` and an ``exclaim: (s: string) => string`` function into a ``shout`` function.
3. Explain (briefly, as a comment) why the generic parameters ``A``, ``B``, ``C`` are each necessary rather than using a single type parameter.

TYPESCRIPT

MEDIUM

Result Type For Division

Rather than throwing exceptions for expected failure cases (like division by zero), your team wants to adopt a Rust-style `Result<T, E>` pattern so callers are forced by the type system to handle both success and failure explicitly.

```
type Result<T, E = string> = /* TODO: discriminated union with an `ok` boolean */;

function divide(a: number, b: number): Result<number> {
  // TODO: implement, returning an error Result instead of throwing on division by zero
}
```

Your task:

1. Define `Result<T, E = string>` as a discriminated union with an `ok: true` variant holding `value: T` and an `ok: false` variant holding `error: E`.
2. Implement `divide` to return an error Result (not throw) when `b === 0`, and a success Result otherwise.
3. Write a `handleResult(result: Result<number>): string` function that narrows on `result.ok` to safely access `value` or `error`.

MEDIUM

In-Memory Entity Repository

Multiple domain objects in your app (users, products, orders) all need basic CRUD storage during tests, before a real database is wired up. You want a single generic repository implementation that works for any entity type as long as it has an `id`.

```
interface Entity {
    id: string;
}

interface Repository<T extends Entity> {
    findById(id: string): T | undefined;
    save(entity: T): void;
    delete(id: string): boolean;
    findAll(): T[];
}

class InMemoryRepository<T extends Entity> implements Repository<T> {
    private items = new Map<string, T>();
    // TODO: implement all Repository methods
}
```

Your task:

1. Implement all four `Repository<T>` methods on `InMemoryRepository`.
2. `save` should upsert using the entity's own `id` as the map key.
3. `delete` should return `true` if an entity was removed, `false` if the id didn't exist.

MEDIUM

Typed Publish-Subscribe Channel

Different parts of your app need to react to 'order placed' events without being tightly coupled. You want a reusable `Channel<T>` class (usable for any single event payload type) where subscribers can cleanly unsubscribe later, e.g. when a UI component unmounts.

```
type Handler<T> = (payload: T) => void;
type Unsubscribe = () => void;

class Channel<T> {
  private handlers: Set<Handler<T>> = new Set();

  subscribe(handler: Handler<T>): Unsubscribe {
    // TODO: implement, returning a function that removes this handler
  }

  publish(payload: T): void {
    // TODO: implement
  }
}
```

Your task:

1. Implement ``subscribe`` to register the handler and return an ``Unsubscribe`` function that removes it from the set.
2. Implement ``publish`` to invoke every currently-registered handler with the payload.
3. Demonstrate usage with a ``Channel<{ orderId: string; total: number }>`` and show calling the returned unsubscribe function.

MEDIUM

Typed Group By Utility

You constantly need to group arrays of objects by some derived key — grouping orders by status, users by role, etc. You want one reusable, fully generic `groupBy`` utility instead of writing this loop over and over.

```
function groupBy<T, K extends string | number>(items: T[], keyFn: (item: T) => K): Record<K, T[]> {
    // TODO: implement
}
```

Your task:

1. Implement `groupBy` to bucket items into a `Record<K, T[]>` using the result of `keyFn(item)` as the bucket key.
2. Initialize each bucket array lazily the first time a given key is encountered.
3. Demonstrate usage grouping a small array of `{ status: string }` objects by `status`.

TYPESCRIPT

MEDIUM

Typed HTTP Client

You're defining the contract for an HTTP client abstraction used across the app, so that different implementations (real fetch-based, mocked for tests) can be swapped freely while every call site still gets full type safety on request and response bodies.

```
interface HttpClient {
  get<T>(url: string): Promise<T>;
  post<T, B = unknown>(url: string, body: B): Promise<T>;
}

interface CreateOrderRequest {
  productId: string;
  quantity: number;
}

interface OrderResponse {
  orderId: string;
  status: string;
}

async function createOrder(client: HttpClient, request: CreateOrderRequest): Promise<OrderResponse> {
  // TODO: implement using client.post with explicit type arguments
}
```

Your task:

1. Implement `createOrder` using `client.post` with explicit generic type arguments for both the response and body types.
2. Ensure the function's own return type (`Promise<OrderResponse>`) lines up with the type argument passed to `post`.
3. Explain briefly why `B = unknown` (rather than `any`) is a safer default for the body type parameter.

MEDIUM

Public User DTO Transform

Your database layer returns full user rows including a hashed password, but API responses must never include that field. You want a type-safe transform function whose return type makes it structurally impossible to leak `passwordHash` to a client.

```
interface DbUser {
  id: string;
  name: string;
  email: string;
  passwordHash: string;
  createdAt: Date;
}

type PublicUser = /* TODO: derive from DbUser, excluding passwordHash */;

function toPublicUser(user: DbUser): PublicUser {
  // TODO: implement
}
```

Your task:

1. Define `PublicUser` using a built-in utility type that derives it from `DbUser` while excluding `passwordHash`.
2. Implement `toPublicUser` to strip `passwordHash` and return the remaining fields.
3. Do this without manually re-listing every remaining field name in the type definition.

MEDIUM

Typed Debounce Function

A search-as-you-type input fires an API call on every keystroke, overwhelming the backend. You need a generic `debounce` higher-order function that wraps ANY function (preserving its exact parameter types) and delays invocation until the user pauses typing.

```
function debounce<Args extends unknown[]>(  
  fn: (...args: Args) => void,  
  waitMs: number  
) : (...args: Args) => void {  
  // TODO: implement  
}
```

Your task:

1. Implement ``debounce`` so repeated calls within ``waitMs`` of each other cancel the pending invocation and reschedule it.
2. Preserve the wrapped function's exact parameter types via the ``Args`` generic (no ``any``).
3. Demonstrate usage by debouncing a ``search(query: string): void`` function.

MEDIUM

Middleware Pipeline

You're building a tiny Express-like middleware system for an internal tool: each middleware receives a shared context object and a ``next`` callback, and can choose whether to call ``next()`` to continue the chain. You need this fully typed against the context shape.

```
type Middleware<T> = (context: T, next: () => void) => void;

class MiddlewarePipeline<T> {
  private middlewarees: Middleware<T>[] = [];

  use(middleware: Middleware<T>): this {
    // TODO: implement
  }

  execute(context: T): void {
    // TODO: implement, running middlewarees in registration order
  }
}
```

Your task:

1. Implement `use` to register a middleware and return `this` for chaining.
2. Implement `execute` to run middlewares in the order they were registered, where each middleware controls advancement by calling `next()`.
3. Ensure that if a middleware never calls `next()`, the chain correctly stops (later middlewares do not run).

